

safety and eliminates the errors of manual parsing.

State management: `Data<T>` provides a thread-safe way of sharing application state between multiple worker threads by means of internal reference counting (Arc).

HttpServer: Starts and controls worker threads.

App: Defines routes and middleware.

Service: Contains actual request-response logic.

Request lifecycle: Together they form a pipeline that processes every request.

Building the API: Implementation phase

Now that you have Actix Web project setup, we can build a clean, fast API. This is all about structuring endpoints, error handling and adding middleware without losing current performance.

Endpoint definition (CRUD with zero cost abstractions):

Actix Web can define endpoints as simple async functions. Rust's zero-cost abstractions mean that these handlers compile down to efficient machine code, with no runtime overhead.

```
use actix_web::{get, post, put, delete, web, HttpResponse,
Responder};
#[get("/items")]
async fn list_items() -> impl Responder {
    HttpResponse::Ok().body("All items")
}

#[post("/items")]
async fn create_item() -> impl Responder {
    HttpResponse::Created().body("Item created")
}

#[put("/items/{id}")]
async fn update_item(id: web::Path<u32>) -> impl Responder {
    HttpResponse::Ok().body(format!("Updated item {}", id))
}

#[delete("/items/{id}")]
async fn delete_item(id: web::Path<u32>) -> impl Responder {
    HttpResponse::Ok().body(format!("Deleted item {}", id))
}
```

Routes are also type-safe, declarative, and fast. Abstraction is not a hidden cost — what you write is close to what runs.

Error handling (thiserror + anyhow): Rust encourages explicit error handling. Instead of exceptions, use `Result<T, E>`. With `thiserror`, you define structured, readable errors:

```
use thiserror::Error;
#[derive(Error, Debug)]
```

```
pub enum ApiError {
    #[error("Resource not found")]
    NotFound,
    #[error("Internal error")]
    Internal,
}
```

For flexible error propagation (especially in services), `anyhow` is useful:

```
use anyhow::Result;
fn process_logic() -> Result<String> {
    Ok("Success".into())
}
```

In Actix handlers, returning `Result` keeps logic clean and composable. Errors can later be mapped to HTTP responses centrally, avoiding repetitive code.

Middleware design (fast and non-blocking): Middleware in Actix runs alongside the request lifecycle without blocking execution. You can stack multiple layers like logging, authentication, and headers efficiently.

```
use actix_web::{middleware, App};
App::new()
    .wrap(middleware::Logger::default())
```

You can also inject headers globally:

```
use actix_web::middleware::DefaultHeaders;
.wrap(DefaultHeaders::new().add(("X-Version", "1.0")))
```

Actix Web lets you scale endpoints, manage errors robustly, and extend functionality via middleware—all while maintaining the raw performance Rust is known for.

Database integration: Low-latency strategies

Compile-time safety (SQLx vs Diesel): Rust really works well when you push correctness to compile time. With `SQLx` you get compile-time checked queries, meaning your SQL is checked against database schema before your code even runs.

```
use sqlx::query;
let rec = query!("SELECT id, name FROM users WHERE id = $1",
1)
    .fetch_one(&pool)
    .await?;
```

And compilation fails if the query is bad or doesn't match. This eliminates a whole class of runtime bugs. On the other hand, `Diesel` provides a more traditional ORM experience with a Rust-based query builder: